

# Feature Extraction and Object Tracking for a Descending Parachute Sequence

Edosa Ebohon  
30436293

*MSc in Robotics and Artificial Intelligence  
University of Lincoln*

**Abstract**—This paper presents a feature extraction and object tracking pipeline applied to a 51-frame sequence of a descending parachute. Task 1 extracts and analyses shape and Histogram of Oriented Gradients (HoG) features from ground-truth binary masks, identifying the most temporally discriminative descriptors. Task 2 implements two Kalman filters from scratch — one for centroid translation, one for orientation — using data-derived noise parameters estimated from training frames 0–40 only. The translation filter achieves a test RMS of 3.139 px and the rotation filter 1.072° across frames 41–50, both within 1–2% of their respective signal ranges.

## I. INTRODUCTION

This work presents a feature extraction and object tracking process implemented on a 51-frame descending parachute sequence with a Ground-Truth (GT) binary mask provided for each frame as the primary data source.

Two tasks act independently. Task 1 extracts shape descriptors [1] and Histogram of Oriented Gradients (HoG) features [2] from each GT mask and analyses how they change across the sequence. Task 2 implements two Kalman filters [3] from scratch — one tracking the parachute’s 2D centroid, the other tracking its orientation angle — both based on a constant-velocity motion model trained on frames 0–40. Predictions are generated for frames 41–50 without ground truth and compared against true measurements to evaluate performance.

To prevent evaluation bias, five frames were quarantined prior to feature development using a seeded RNG (seed = 42; indices: 8, 20, 29, 36, 48), ensuring reproducibility. These frames were excluded from all development-time inspection. For the Kalman task, all noise parameters were derived from training frames 0–40 only; frames 41–50 were not inspected at any point during development.

## II. TASK 1: FEATURE EXTRACTION

### A. Shape Features

The objective is to characterise the parachute’s silhouette geometry across 51 frames and assess whether any descriptor captures a consistent pattern of change during descent.

1) **Methodology:** Shape descriptors operate on the binary silhouette rather than internal appearance, making them invariant to illumination and texture variation — both present in aerial footage [1]. Four descriptors were selected to capture the two geometric properties most likely to change as a parachute descends and rotates: compactness and elongation. All are independent of object position and size.

- **Solidity** ( $\text{Area}/\text{ConvexArea}$ ) — a fully deployed canopy is expected to be largely convex. Solidity measures deviation from convexity [1]; any significant drop indicates canopy deformation or partial occlusion by rigging, providing a structural signal not recoverable from the remaining three descriptors.
- **Circularity** ( $4\pi \cdot \text{Area}/\text{Perimeter}^2$ ) — a face-on parachute approximates a circle. The isoperimetric inequality guarantees this ratio is maximised by a circle and decreases as the silhouette departs from circular [1], making it theoretically well-motivated as a measure of rotation away from face-on.
- **Non-compactness** ( $1 - \text{Circularity}$ ) — by construction the complement of circularity, carrying no independent geometric information — confirmed empirically in Section II-C.
- **Eccentricity** — captures elongation via second-order central moments rather than the perimeter estimate [1]. Moment-based measures integrate over the entire foreground region, making them robust to boundary-pixel discretisation that amplifies noise in  $\text{Perimeter}^2$ -based descriptors.

The GT mask is preprocessed with `bwareafilt` to retain the largest connected component, eliminating isolated noise without a tuned threshold. If no foreground pixels exist, NaN is returned to avoid corrupting downstream statistics. Features are extracted via `regionprops`. Two robustness guards are applied to circularity: a division-by-zero check when  $\text{Perimeter} = 0$ , and a clamp at 1 to enforce the theoretical upper bound against numerical overshoot from MATLAB’s discrete perimeter estimator.

2) **Results:** Feature trajectories across all 51 frames are shown in Figure 1. Summary statistics are given in Table I.

TABLE I: Shape Feature Summary Statistics: Development vs Quarantine Set.

| Feature         | Dev Mean | Dev Std | Qtn Mean | Qtn Std |
|-----------------|----------|---------|----------|---------|
| Solidity        | 0.9502   | 0.0136  | 0.9528   | 0.0115  |
| Non-compactness | 0.2135   | 0.0209  | 0.2181   | 0.0210  |
| Circularity     | 0.7865   | 0.0209  | 0.7819   | 0.0210  |
| Eccentricity    | 0.8586   | 0.0187  | 0.8613   | 0.0203  |

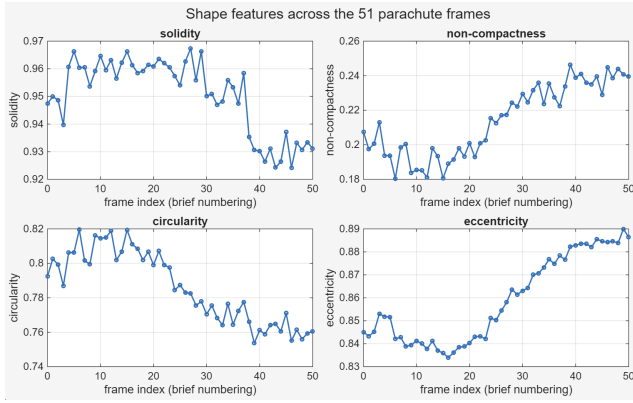


Fig. 1: Shape feature trajectories across 51 frames.

3) **Discussion:** Table I and Figure 1 reveal two distinct patterns. Eccentricity and circularity both capture progressive canopy elongation as the parachute rotates away from face-on. Our results show circularity achieves a stronger monotonic trend ( $|\rho| = 0.8554$  vs  $0.8166$  for eccentricity), consistent with the isoperimetric inequality [1]: as the silhouette departs from circular, circularity decreases directly. However, circularity’s dependence on  $\text{Perimeter}^2$  amplifies raster boundary noise, producing a visibly noisier trajectory in Figure 1. Eccentricity, derived from area integrals [1], provides a smoother and more stable estimate of the same geometric change. Where trend strength is the priority, our results indicate circularity is the stronger descriptor; where trajectory stability matters, eccentricity is preferable. Non-compactness shares circularity’s  $|\rho|$  identically — it is its algebraic restatement, not an independent descriptor.

Solidity captures a geometrically distinct property [1] and our measurements show it remains stable (mean  $0.9502 \pm 0.0136$ ) until a sharp drop near frame 37 — consistent with rigging lines or canopy folds entering the silhouette — an event not visible in Table I’s summary statistics but clearly present in Figure 1. Quarantine means lie within 0.5 SD of development means across all four features, confirming the observed patterns generalise beyond the development set.

### B. HoG Features

The objective is to extract a per-frame texture descriptor capturing the directional distribution of edge energy within the parachute region, and to determine whether any orientation bin shows a systematic pattern across the sequence.

1) **Methodology:** HoG was selected because gradient-based descriptors normalise out additive illumination shifts [2] — a necessary property for outdoor aerial sequences. Unlike shape features, HoG captures internal edge structure rather than silhouette geometry, providing evidence of rotation independent of Section II-A.

Each frame is converted to grayscale and background pixels are zeroed using the GT mask, ensuring gradients reflect only parachute edges. The image is cropped to the parachute bounding box, making all spatial measurements object-relative.

Gradients are computed using the  $[-1, 0, 1]$  kernel, a first-order derivative operator sufficient for dominant edge orientations [2]. Higher-order kernels introduce smoothing that suppresses the fine panel-seam edges carrying the rotation signal. Unsigned orientations over  $[0^\circ, 180^\circ)$  are used since edge polarity is unstable under changing illumination, whereas orientation is not [2].

Hard binning assigns gradient magnitudes to four  $45^\circ$ -wide bins ( $0^\circ, 45^\circ, 90^\circ, 135^\circ$ ). Soft binning is excluded because at four bins the quantisation difference is negligible and hard binning introduces no additional constants requiring justification.

Cell size is defined by a  $4 \times 4$  grid over the parachute bounding box. A fixed pixel cell size would represent a different fraction of the canopy at different apparent scales, making cross-frame comparisons unreliable. The  $4 \times 4$  division follows the object-tiling principle established in [2], fine enough to preserve spatial layout while coarse enough for stable gradient statistics per cell. A minimum cell size of 2 px is enforced — a 4-bin histogram built from fewer than approximately 4 gradient samples carries no meaningful statistical content. Across frames, cell sizes range from 11–21 px (median 18), and feature lengths from 64–112 (median 80). Global  $L_2$  normalisation removes residual contrast variation between frames [2]; an epsilon guard prevents division-by-zero.

2) **Results:** Per-bin energy trajectories are shown in Figure 2. Summary statistics are given in Table II.

TABLE II: HoG Per-Bin Energy Summary Statistics, Development vs. Quarantine Set.

| Bin         | Dev Mean | Dev Std | Qtn Mean | Qtn Std |
|-------------|----------|---------|----------|---------|
| $0^\circ$   | 1.6722   | 0.2888  | 1.6228   | 0.2881  |
| $45^\circ$  | 1.5302   | 0.2131  | 1.4678   | 0.2804  |
| $90^\circ$  | 1.6543   | 0.5785  | 1.5678   | 0.3342  |
| $135^\circ$ | 1.4251   | 0.3083  | 1.4673   | 0.3244  |

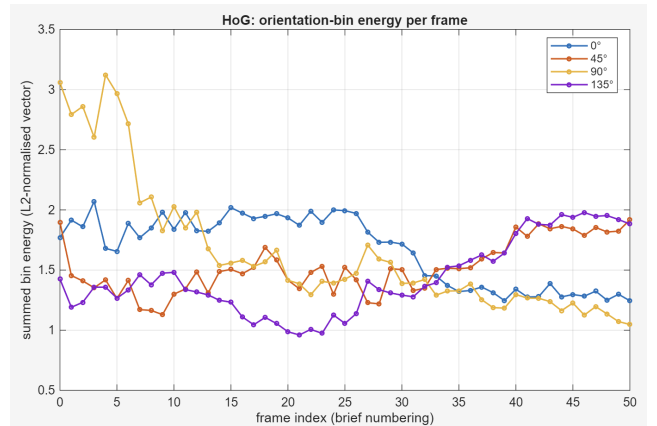


Fig. 2: HoG per-bin energy trajectories across 51 frames.

3) **Discussion:** Per-bin energy trajectories are shown in Figure 2 and summarised in Table II. Our results confirm

a systematic energy migration from the  $90^\circ$  bin towards the diagonal bins across the sequence. The  $90^\circ$  bin begins dominant ( $\sim 3.0$  in early frames) and decays to  $\sim 1.3$  by frame 35, while the  $45^\circ$  and  $135^\circ$  bins rise from  $\sim 1.4$  to  $\sim 1.9$  by frame 40, with a crossover near frame 33.

This migration is physically consistent with the canopy rotating away from face-on: as the panel seams that initially run predominantly vertically tilt into diagonal orientations, gradient energy shifts from the  $90^\circ$  bin into the diagonal bins. The  $90^\circ$  bin’s standard deviation (SD = 0.5785, vs  $\leq 0.3083$  for all others, Table II) confirms it as the most temporally informative—high variance across the sequence reflects sensitivity to the rotation being measured, not noise. The  $0^\circ$  bin is the least informative: a stable mean and the smallest standard deviation indicate no directional response to the descent.

Quarantine means agree with development within 0.09 across all four bins (Table II). The larger SD gap for  $90^\circ$  between development (0.5785) and quarantine (0.3342) reflects which high-energy frames each subset captured rather than a pipeline inconsistency. This energy migration independently confirms the rotation identified through silhouette geometry in Section II-A—two geometrically distinct feature families detecting the same physical event provides mutual validation of both measurements.

### C. Discriminative Analysis

The objective is to quantitatively determine which feature family better represents a descending parachute and to identify the most temporally informative individual descriptors.

1) **Methodology:** Two complementary measures were applied to all eight features on the development set and confirmed on the quarantine set:

- **CoV** =  $\sigma/|\mu|$ : a unitless measure of relative variability, preferred over variance because feature scales differ; it enables direct cross-family comparison without normalisation assumptions.
- **Spearman**  $|\rho|$  with frame index: a non-parametric measure of monotonic trend strength, chosen over Pearson because several feature trajectories in Sections II-A and II-B are non-linear.

Both were computed on the development set and independently verified on the quarantine set. To further test ranking stability, 10 independent random 5-frame subsamples were drawn across different RNG seeds and the strongest feature per family recorded across all 10 draws.

2) **Results:** Results are given in Tables III and IV and visualised in Figure 3. Across the shape family, our results show circularity achieves the strongest trend ( $|\rho| = 0.8554$ ), with eccentricity close behind ( $|\rho| = 0.8166$ ). Solidity is the weakest ( $|\rho| = 0.5944$ ), reflecting its step-change rather than monotonic behaviour.

Across the HoG family, our results show the  $90^\circ$  bin dominates with the strongest trend of any feature ( $|\rho| = 0.9449$ ) and the highest relative variability (CoV = 0.3497)—approximately  $16\times$  the CoV of eccentricity and  $13\times$

that of circularity. The  $0^\circ$ ,  $45^\circ$ , and  $135^\circ$  bins trail considerably ( $|\rho|$  ranging 0.61–0.73). Non-compactness shares circularity’s  $|\rho|$  identically—it is defined as  $1 - \text{circularity}$ , making it a restatement rather than an independent descriptor.

All quarantine shape  $|\rho|$  values reduce to 0.9000—a consequence of the 5-frame sample compressing rank resolution. The multi-seed test confirms the ranking is stable: HoG  $90^\circ$  wins 7 of 10 subsamples and circularity wins 6 of 10.

TABLE III: CoV and Spearman  $|\rho|$  for all eight features, development vs quarantine set.

| Feature         | CoV (dev) | CoV (qtn) | $ \rho $ (dev) | $ \rho $ (qtn) |
|-----------------|-----------|-----------|----------------|----------------|
| Solidity        | 0.0143    | 0.0121    | 0.5944         | 0.9000         |
| Non-compactness | 0.0981    | 0.0964    | 0.8554         | 0.9000         |
| Circularity     | 0.0266    | 0.0269    | 0.8554         | 0.9000         |
| Eccentricity    | 0.0218    | 0.0235    | 0.8166         | 0.9000         |
| HoG $0^\circ$   | 0.1727    | 0.1775    | 0.7293         | 0.8000         |
| HoG $45^\circ$  | 0.1393    | 0.1910    | 0.6500         | 0.7000         |
| HoG $90^\circ$  | 0.3497    | 0.2132    | 0.9449         | 1.0000         |
| HoG $135^\circ$ | 0.2163    | 0.2211    | 0.6146         | 0.7000         |

TABLE IV: Multi-seed robustness: family winner frequency across 10 independent 5-frame subsamples. Wins attributed to circularity as the meaningful descriptor; non-compactness always shares its rank as its mathematical complement.

| Family | Feature        | Wins (of 10) |
|--------|----------------|--------------|
| Shape  | Circularity    | 6            |
| Shape  | Eccentricity   | 2            |
| Shape  | Solidity       | 2            |
| HoG    | $90^\circ$ bin | 7            |
| HoG    | $0^\circ$ bin  | 3            |

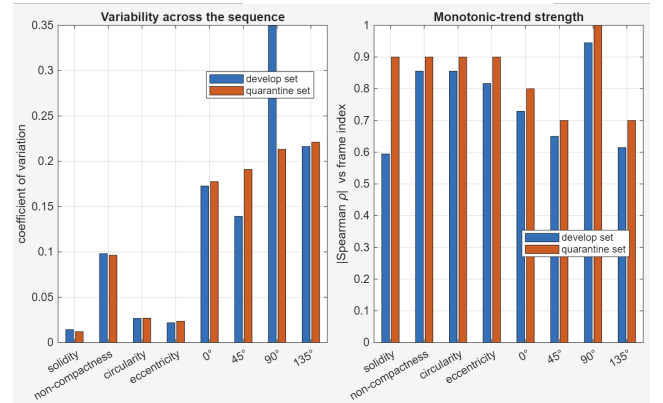


Fig. 3: Grouped bar chart of CoV and Spearman  $|\rho|$  for all eight features, development vs quarantine set.

3) **Discussion:** Table III confirms no single feature dominates all criteria in our experiment. For monotonic temporal tracking, our results show HoG  $90^\circ$  is the strongest descriptor ( $|\rho| = 0.9449$ , CoV = 0.3497). Among shape features, our results show circularity achieves the strongest trend ( $|\rho| = 0.8554$ ) as the most temporally discriminative

shape descriptor. Eccentricity ( $|\rho| = 0.8166$ ) is weaker in trend strength but produces a more stable trajectory — evidenced in Figure 1 — because its moment-based derivation [1] is less sensitive to boundary discretisation than circularity’s Perimeter<sup>2</sup> formulation. The choice between them therefore depends on whether trend strength or trajectory stability is the priority for the application.

The two feature families are complementary rather than interchangeable. Shape features describe silhouette geometry [1] and remain informative regardless of canopy appearance — a uniform canopy would collapse the HoG rotation signal [2] while leaving eccentricity and solidity unaffected. HoG captures orientation information unavailable to shape features [2]. Both families independently identify the same rotation event in this sequence, reinforcing each other. For a single discriminative feature, our results indicate HoG 90° is optimal ( $|\rho| = 0.9449$ ). Within the shape family, circularity is the strongest descriptor by trend strength; eccentricity should be included where trajectory stability is the priority, as evidenced by its smoother progression in Figure 1.

### III. TASK 2: OBJECT TRACKING

#### A. Measurements from GT

The objective is to extract per-frame centroid and orientation from GT masks as observation inputs for the Kalman filters in Sections III-B and III-C.

1) **Methodology:** For each frame, `bwareafilt` retains only the largest connected component of the GT mask, removing stray pixels without a tuned threshold. If no foreground pixels exist, NaN is returned to avoid corrupting filter initialisation.

The centroid is computed as the mean of foreground pixel coordinates — the minimum-variance unbiased estimator of centre of mass for a uniform binary region [1] — making it the appropriate choice for a filled silhouette mask.

Orientation is obtained via PCA of foreground pixel coordinates [4]. After mean-centring, the  $2 \times 2$  covariance matrix is eigendecomposed and the principal eigenvector defines the dominant axis [4]. Its angle from the positive x-axis is  $\theta = \text{atan2d}(v_y, v_x)$ .

Since PCA yields an axis,  $\theta$  and  $\theta + 180^\circ$  are geometrically identical [4]. The raw `atan2d` output is first folded to  $(-90^\circ, +90^\circ]$ . Sequential unwrapping then resolves inter-frame sign flips: if  $|\theta_t - \theta_{t-1}| > 90^\circ$ ,  $\theta_t$  is adjusted by  $\pm 180^\circ$ . MATLAB’s `unwrap()` was not used — it assumes  $2\pi$  periodicity, whereas this ambiguity is  $180^\circ$ -periodic. The  $90^\circ$  threshold is the geometric midpoint of the ambiguity and is not tuned.

2) **Results:** Key measurement ranges are summarised in Table V and trajectories shown in Figure 4.  $c_x$  shows near-linear growth,  $c_y$  increases more slowly with a visible slope change near frame 30, and  $\theta$  increases smoothly from  $82.2^\circ$  to  $156.9^\circ$  ( $\sim 1.5^\circ/\text{frame}$ ). One unwrap correction occurs between frames 3–4.

TABLE V: Centroid and Orientation Measurement Summary Across 51 Frames.

| Measurement                     | Start | End   | Rate                          |
|---------------------------------|-------|-------|-------------------------------|
| $c_x$ (px)                      | 101.4 | 303.4 | $\sim 4$ px/frame             |
| $c_y$ (px)                      | 91.3  | 127.4 | $\sim 0.7$ px/frame           |
| $\theta$ unwrapped ( $^\circ$ ) | 82.2  | 156.9 | $\sim 1.5^\circ/\text{frame}$ |

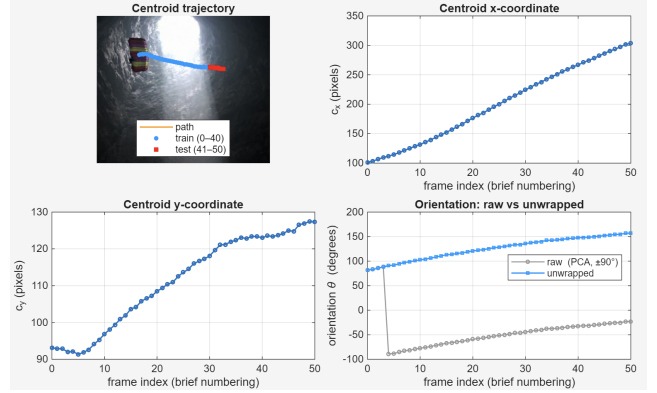


Fig. 4: (a) Centroid trajectory on frame 0 with training and test frames distinguished, (b)  $c_x$  vs frame index, (c)  $c_y$  vs frame index, (d) raw vs unwrapped orientation across 51 frames.

3) **Discussion:** The near-linear  $c_x$  trajectory confirms the constant-velocity assumption holds in  $x$  — the model’s prediction of uniform horizontal displacement is consistent with the measured data. The  $c_y$  slope change near frame 30 is a partial violation of the same assumption: a constant-velocity model fitted over frames 0–40 will learn an averaged  $v_y$  that is systematically too low after frame 30, meaning every open-loop prediction step in frames 41–50 compounds this underestimate. This is not a filter implementation issue but a fundamental consequence of applying a linear motion model [3] to a non-linear signal.

The single unwrap correction across 50 transitions confirms the  $90^\circ$  threshold is geometrically appropriate — a more permissive threshold would introduce false corrections, while a stricter one would miss genuine ambiguity flips. The smooth  $74.7^\circ$  total rotation at  $\sim 1.5^\circ/\text{frame}$  with no slope change across the training window means the constant-angular-velocity assumption is well supported empirically. This makes the rotation filter’s motion model a better fit to the underlying dynamics than the translation filter’s from the outset.

#### B. Translation Kalman Filter

The objective is to implement a Kalman filter from first principles [3] tracking the parachute’s 2D centroid, trained on frames 0–40 and evaluated by open-loop prediction on frames 41–50.

1) **Methodology:** The Kalman filter models motion as a linear dynamical system under Gaussian noise [3], cycling

between a predict step and an update step. On test frames 41–50, only prediction runs; no measurements are consumed.

**State Vector:**  $\mathbf{x} = [c_x, c_y, v_x, v_y]^\top$ . Velocity components are latent states required for forward projection under the constant-velocity assumption [3].

**State Transition Matrix ( $F$ ):** encodes the constant-velocity model with  $dt = 1$  frame:

$$F = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$F$  is analytically determined by the motion model and contains no tuned values.

**Measurement Matrix ( $H$ ):**

$$H = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}$$

maps state to observable centroid position.  $H$  is analytically determined.

**Measurement Noise Covariance ( $R$ ):** estimated from training residuals. Linear fits to  $c_x$  and  $c_y$  over frames 0–40 yield residual variances  $R = \text{diag}([11.85, 2.75]) \text{ px}^2$ , reflecting actual centroid measurement uncertainty rather than an assumed value.

**Process Noise Covariance ( $Q$ ):** derived from the discrete-time constant-velocity white-noise-acceleration model [3]. Empirical frame-to-frame velocity changes give  $\sigma_{ax}^2, \sigma_{ay}^2$ , which parameterise:

$$Q_{axis} = \sigma_a^2 \cdot \begin{bmatrix} dt^4/4 & dt^3/2 \\ dt^3/2 & dt^2 \end{bmatrix}$$

Separate  $\sigma_a^2$  values for  $x$  and  $y$  reflect the greater velocity variation in  $y$  from the slope change near frame 30.  $Q$  is assembled via `blkdiag` and reordered by permutation  $[1, 3, 2, 4]$  to match the declared state order  $[c_x, c_y, v_x, v_y]$  — without this step,  $Q$  entries would be silently misaligned with  $F$  and  $H$ .

**Initialisation:**  $\mathbf{x}_0 = [101.43, 93.17, 1.96, -0.31]^\top$  from the frame-0 centroid and a two-frame velocity difference.  $P_0$  uses training-set variances to encode initial uncertainty.

The filter is implemented as two shared functions — `kfPredict` and `kfUpdate` — used by both filters, ensuring an identical, auditable implementation. The Joseph-form covariance update,  $P = (I - KH)P(I - KH)^\top + K RK^\top$ , is algebraically equivalent to the standard form but preserves positive-semidefiniteness under floating-point rounding [3], preventing numerical divergence.

**2) Results:** Performance is shown in Figure 5 and summarised in Table VI. Per-frame errors are reported in Section III-D.

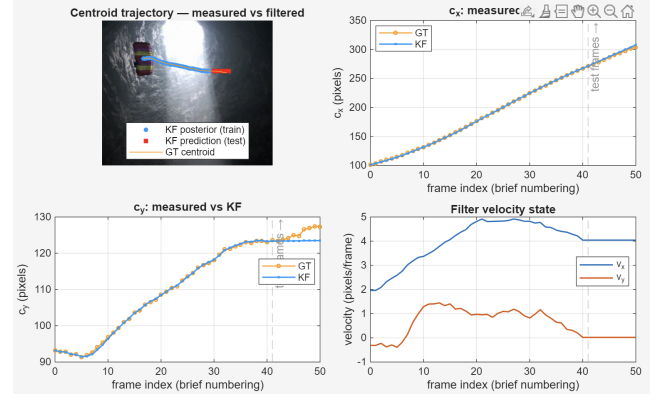


Fig. 5: (a) Centroid trajectory with train/test frames marked, (b)  $c_x$  GT vs KF, (c)  $c_y$  GT vs KF, (d) velocity states  $v_x, v_y$ . Dashed line marks the train/test boundary.

**3) Discussion:** Table VI shows a training RMS of 0.49 px — the filter’s posterior estimates closely track GT centroids because the update step actively corrects each prediction using the incoming measurement, preventing error accumulation. The test RMS of 3.14 px represents a six-fold degradation from training, the direct consequence of the update step being absent: without measurement corrections, any bias in the learned velocity compounds with each prediction step rather than being continuously corrected.

The monotonically growing error pattern — 0.447 px at frame 41 rising to 5.860 px at frame 50 (Table VII) — is the diagnostic signature of this compounding bias rather than random noise, which would produce an irregular pattern with no directional trend. The source is the  $c_y$  slope change near frame 30: the constant-velocity model [3] cannot distinguish between the pre- and post-frame-30 dynamics and fits a single averaged  $v_y$  across both, which systematically underestimates the true descent rate in the test window. The  $c_x$  predictions remain more accurate throughout because the horizontal trajectory is genuinely near-linear across all 51 frames — the constant-velocity assumption holds in  $x$  in a way it does not in  $y$ , confirming the limitation is signal-specific rather than a general filter failure.

### C. Rotation Kalman Filter

The objective is to track the parachute’s orientation angle using a Kalman filter from first principles [3], trained on frames 0–40 and evaluated by open-loop prediction on frames 41–50.

**1) Methodology:** The same `kfPredict` and `kfUpdate` functions from Section III-B are used here, adapted to a two-state angular representation.

**State Vector:**  $\mathbf{x} = [\theta, \omega]^\top$ , where  $\omega$  is angular velocity in degrees per frame — a latent state required to propagate  $\theta$  under the constant-angular-velocity model [3].

**State Transition Matrix ( $F$ ):**

$$F = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$$



meaning  $\theta' = \theta + \omega$  and  $\omega' = \omega$ .  $F$  is analytically determined. The constant-angular-velocity assumption is well supported empirically — the unwrapped  $\theta$  trajectory rises at  $\sim 1.5^\circ/\text{frame}$  with no visible slope change across the training window (Section III-A).

**Measurement Matrix ( $H$ ):**  $H = \begin{bmatrix} 1 & 0 \end{bmatrix}$  — GT provides  $\theta$  only;  $\omega$  is never measured.  $H$  is analytically determined.

**Measurement Noise Covariance ( $R$ ):** a linear fit to  $\theta$  over frames 0–40 yields residual variance  $R = 3.71 \text{ deg}^2$ , reflecting PCA orientation jitter around an otherwise smooth trend.

**Process Noise Covariance ( $Q$ ):** same discrete-time white-noise-acceleration structure as Section III-B [3], scalar version:

$$Q = \sigma_\alpha^2 \cdot \begin{bmatrix} 1/4 & 1/2 \\ 1/2 & 1 \end{bmatrix}$$

where  $\sigma_\alpha^2 = 1.10 \text{ deg}^2/\text{frame}^2$  is the empirical angular-acceleration variance from training frames. Fully data-derived; no manual tuning applied.

**Initialisation:**  $\mathbf{x}_0 = [82.15^\circ, 0.911^\circ/\text{frame}]^\top$  from the frame-0 orientation and a two-frame angular difference.  $P_0$  is the diagonal of training-set variances. The Joseph-form update is applied via `kfUpdate`, guaranteeing positive-semidefiniteness of  $P$  throughout [3].

2) **Results:** Performance is shown in Figure 6 and summarised in Table VI.

TABLE VI: Kalman Filter RMS Errors, Training vs Test.

| Filter      | Train RMS | Test RMS |
|-------------|-----------|----------|
| Translation | 0.49 px   | 3.14 px  |
| Rotation    | 0.27°     | 1.07°    |

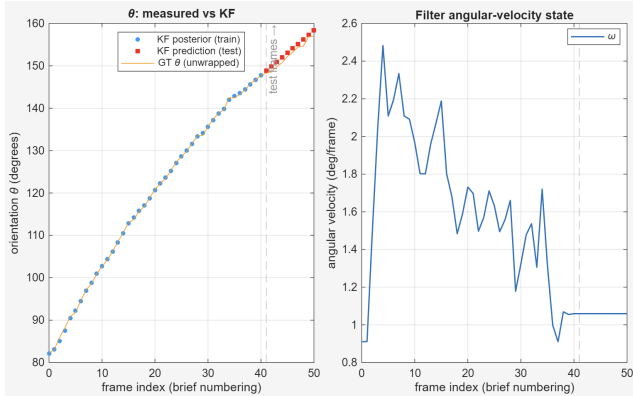


Fig. 6: (a)  $\theta$  GT vs KF posterior (training) and prediction (test) with dashed boundary, (b) angular velocity state  $\omega$  across all frames.

3) **Discussion:** Table VI shows a training RMS of  $0.27^\circ$ , confirming effective measurement assimilation — the update step continuously corrects predictions against incoming GT orientation measurements. The test RMS of  $1.07^\circ$  reflects open-loop degradation once measurement updates cease.

Figure 6(b) shows  $\omega$  settling and remaining stable through the test window, with a final learned value of  $\sim 1.05^\circ/\text{frame}$

against an observed test-window rate of  $\sim 0.9^\circ/\text{frame}$  — a small positive bias. Critically, Figure 6(a) shows no monotonic error growth across frames 41–50. The flat error pattern indicates the  $\omega$  bias is insufficient to produce compounding drift over 10 prediction steps. The constant-angular-velocity assumption [3] is well supported by the near-linear  $\theta$  trajectory established in Section III-A, which is why the rotation filter does not exhibit the accumulating error seen in the translation filter. This contrasting behaviour is examined jointly in Section III-D.

#### D. Combined Evaluation

The objective is to formally evaluate both filters on test frames 41–50 and compare their error behaviour as a diagnostic of motion model fit quality.

1) **Methodology:** Translation error is the Euclidean distance between predicted and GT centroid per frame:

$$e_{pos}(t) = \sqrt{(c_{x,KF} - c_{x,GT})^2 + (c_{y,KF} - c_{y,GT})^2}$$

Euclidean distance is invariant to error direction — a given displacement magnitude represents the same physical offset regardless of axis alignment, which component-wise errors would not.

Rotation error is the shortest angular distance on the  $180^\circ$ -periodic orientation space, appropriate for an axis where  $\theta$  and  $\theta + 180^\circ$  are physically identical:

$$e_\theta(t) = \min(|\Delta\theta|, 180 - |\Delta\theta|), \quad \Delta\theta = \theta_{KF} - \theta_{GT}$$

Both metrics are computed exclusively on frames 41–50; no ground truth was consumed during this window.

2) **Results:** Per-frame errors are given in Table VII and trajectories in Figure 7. Translation error: mean 2.647 px, RMS 3.139 px, max 5.860 px (frame 50). Rotation error: mean  $1.006^\circ$ , RMS  $1.072^\circ$ , max  $1.581^\circ$  (frame 48).

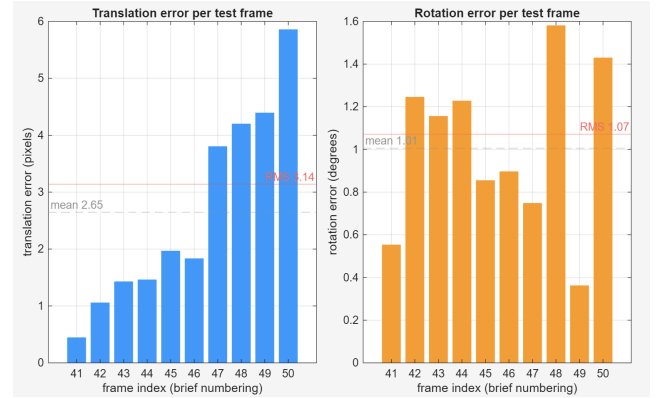


Fig. 7: Per-frame (a) translation error (px) and (b) rotation error ( $^\circ$ ) across frames 41–50, with dashed mean and solid RMS reference lines.

3) **Discussion:** Tables VII and the summary statistics above reveal a clear qualitative distinction. In Figure 7(a), translation error grows steadily from 0.447 px at frame 41 to 5.860 px at frame 50, with a step increase at frame 47 where  $c_y$  accelerates

TABLE VII: Per-frame Kalman filter prediction errors on test frames 41–50.

| Frame | $c_{x,GT}$ | $c_{y,GT}$ | $c_{x,KF}$ | $c_{y,KF}$ | $e_{pos}(px)$ | $\theta_{GT}(^\circ)$ | $\theta_{KF}(^\circ)$ | $e_\theta(^\circ)$ |
|-------|------------|------------|------------|------------|---------------|-----------------------|-----------------------|--------------------|
| 41    | 271.03     | 123.56     | 271.41     | 123.33     | 0.447         | 148.26                | 148.82                | 0.554              |
| 42    | 274.40     | 123.36     | 275.46     | 123.35     | 1.059         | 148.63                | 149.87                | 1.246              |
| 43    | 278.11     | 123.73     | 279.50     | 123.37     | 1.430         | 149.78                | 150.93                | 1.157              |
| 44    | 282.31     | 124.17     | 283.54     | 123.39     | 1.464         | 150.77                | 151.99                | 1.228              |
| 45    | 286.40     | 124.98     | 287.58     | 123.41     | 1.970         | 152.20                | 153.05                | 0.856              |
| 46    | 290.39     | 124.79     | 291.62     | 123.42     | 1.835         | 153.22                | 154.11                | 0.897              |
| 47    | 293.47     | 126.56     | 295.66     | 123.44     | 3.805         | 154.42                | 155.17                | 0.749              |
| 48    | 297.32     | 126.92     | 299.71     | 123.46     | 4.202         | 154.65                | 156.23                | 1.581              |
| 49    | 301.76     | 127.40     | 303.75     | 123.48     | 4.395         | 156.93                | 157.29                | 0.363              |
| 50    | 303.36     | 127.33     | 307.79     | 123.50     | 5.860         | 156.92                | 158.35                | 1.430              |

by 1.77 px against the filter’s fixed  $v_y$  estimate. Rotation errors in Figure 7(b) fluctuate between  $0.363^\circ$  and  $1.581^\circ$  without systematic growth.

This contrast is directly diagnostic of model-fit quality. Growing translation error is the signature of accumulated velocity bias [3] — each step adds a fixed underestimate of  $v_y$ , compounding linearly with prediction steps, tracing back to the  $c_y$  slope change in Section III-A. Flat rotation error confirms no equivalent bias in  $\omega$ . Both filters operate within 1–2% of their respective signal ranges (translation:  $3.139/\sim 200 \approx 1.6\%$ ; rotation:  $1.072^\circ/\sim 75^\circ \approx 1.4\%$ ), confirming comparable model fit relative to scale. The difference in absolute error magnitude reflects signal dynamics, not filter design quality.

#### IV. LIMITATIONS AND GENERALISATION

**Constant-velocity motion model.** Both filters assume the tracked variable evolves at a constant rate [3]. For translation, this is violated by the  $c_y$  slope change near frame 30 — our results show translation errors grow from 0.447 px at frame 41 to 5.860 px at frame 50 (Table VII), with an additional step at frame 47. A constant-acceleration model [3] could absorb this by including acceleration as a latent state, at the cost of additional parameters requiring estimation from the same training window. The rotation filter exhibits the same limitation at a smaller scale — our results show  $\omega = 1.05^\circ/\text{frame}$  overestimates the test-window rate of  $\sim 0.9^\circ/\text{frame}$  — but insufficient to produce visible compounding over 10 steps.

**HoG dependency on internal edge structure.** The energy migration from the  $90^\circ$  bin to diagonal bins (Section II-B) depends entirely on visible panel seams. A monochromatic canopy would collapse all four bins to similar energy regardless of orientation, reducing the  $|\rho| = 0.9449$  observed in our experiment to near zero. Shape features would remain unaffected [1]. HoG [2] cannot be assumed to generalise to uniform-appearance objects without empirical verification.

**Perimeter-based descriptor instability.** Circularity depends on Perimeter<sup>2</sup> from the discrete raster boundary [1]. Small mask irregularities produce disproportionate perimeter variation — not a property of the parachute’s geometry but of the estimator — explaining the noisier circularity trajectory in Figure 1 relative to eccentricity. In lower-resolution or more irregular silhouettes, this instability would increase further, making eccentricity the only reliable elongation descriptor.

#### PCA orientation ambiguity under multiple rotations.

The sequential  $90^\circ$ -threshold unwrapping resolved the ambiguity with one correction across 50 transitions. This works because per-frame rotation ( $\sim 1.5^\circ$ ) is far below the  $90^\circ$  threshold [4]. In sequences with inter-frame rotation exceeding  $90^\circ$ , genuine rotation would be misclassified as an ambiguity flip, corrupting the filter’s measurement input. A multi-hypothesis tracker [3] would be required for high-speed rotation.

**Sensitivity to segmentation quality.** The pipeline assumes GT masks are provided. In deployment, segmentation errors — missed foreground, included background, fragmented regions — propagate into every downstream measurement: centroid, PCA orientation, HoG bounding box, and all four shape descriptors. The `bwareafilt` selection provides partial robustness against small stray regions but cannot recover from systematic segmentation failures.

#### V. CONCLUSION

This work implemented a two-task pipeline for feature extraction and object tracking on a 51-frame descending parachute sequence. Our results show HoG  $90^\circ$  achieved the strongest monotonic trend ( $|\rho| = 0.9449$ ), with circularity established as the strongest shape descriptor by trend strength ( $|\rho| = 0.8554$ ) and eccentricity preferred where trajectory stability is the priority. Both families independently detected the same canopy rotation event, providing mutual validation. Two from-scratch Kalman filters [3] using data-derived parameters achieved test RMS errors of 3.139 px (translation) and  $1.072^\circ$  (rotation) — both within 1–2% of their signal ranges. The contrasting error patterns — growing translation, flat rotation — demonstrate that filter performance is determined by motion model fit rather than implementation quality.

#### REFERENCES

- [1] M. Sonka, V. Hlavac, and R. Boyle, *Image Processing, Analysis, and Machine Vision*, 4th ed. Stamford, CT, USA: Cengage Learning, 2015.
- [2] N. Dalal and B. Triggs, “Histograms of oriented gradients for human detection,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, San Diego, CA, USA, Jun. 2005, pp. 886–893.
- [3] R. E. Kalman, “A new approach to linear filtering and prediction problems,” *Trans. ASME, J. Basic Eng.*, vol. 82, no. 1, pp. 35–45, Mar. 1960.
- [4] G. Strang, *Introduction to Linear Algebra*, 5th ed. Wellesley, MA, USA: Wellesley-Cambridge Press, 2016.

## APPENDIX

### APPENDIX: FULL SOURCE CODE

```
% CMP9135M Computer Vision | Assessment 2
% main.m : feature extraction + object tracking on the parachute dataset
%
% Run-order note: this script is self-contained. Execute from top to
% bottom, or run individual %% cells after STEP 0 has been run once
% (STEP 0 populates the workspace variables that later steps consume).

clc; clear; close all;

% Reproducibility: any stochastic operation (e.g. random quarantine-set
% selection) is seeded so results are identical across runs.
rng(42);

%% =====
% STEP 0: Setup and data loading
% - Locate the parachute dataset (51 RGB + 51 GT masks).
% - Build a robust binary-mask extractor.
% - Declare the train / quarantine / test splits up front, BEFORE
%   writing any feature or filter logic, so that no parameter is
%   chosen by looking at data we later evaluate on.
%% =====

% ---- 0.1 Dataset paths -----
% Adjust dataRoot if your unzipped dataset sits elsewhere.
dataRoot = 'parachute';
rgbDir    = fullfile(dataRoot, 'images');
gtDir     = fullfile(dataRoot, 'GT');

rgbList = dir(fullfile(rgbDir, '*.png'));
gtList  = dir(fullfile(gtDir, '*.png'));

assert(~isempty(rgbList), 'No RGB images found in %s', rgbDir);
assert(~isempty(gtList), 'No GT masks found in %s', gtDir);
assert(numel(rgbList) == numel(gtList), ...
    'RGB (%d) and GT (%d) file counts must match.', ...
    numel(rgbList), numel(gtList));

% Natural-sort by the integer embedded in each filename so that
% "frame_2.png" comes before "frame_10.png" regardless of zero-padding.
rgbList = sortByEmbeddedNumber(rgbList);
gtList  = sortByEmbeddedNumber(gtList);

numFrames = numel(rgbList);
fprintf('Found %d frame pairs in %s\n', numFrames, dataRoot);

% ---- 0.2 Load images and extract binary masks -----
% Images may differ in spatial size across the sequence, so we store them
% in cell arrays rather than a single numeric array.
rgbImgs = cell(numFrames, 1);
gtMasks = cell(numFrames, 1);
```



```

for i = 1:numFrames
    rgbImgs{i} = imread(fullfile(rgbList(i).folder, rgbList(i).name));
    gtRaw      = imread(fullfile(gtList(i).folder,  gtList(i).name));
    gtMasks{i} = extractMask(gtRaw);
end

fprintf('Loaded %d RGB frames and %d GT masks.\n', numFrames, numFrames);

% ---- 0.3 Declare splits (before any analysis) -----
% Brief frames 0{40 <=> MATLAB indices 1{41 (KF training)
% Brief frames 41{50 <=> MATLAB indices 42{51 (KF evaluation)
trainIdx = 1:41;
testIdx  = 42:51;

% Shape/HoG development quarantine: hold out 5 random frames so that no
% feature-extraction parameter is ever tuned on them. Selected here,
% before any feature logic exists, so the choice is independent of results.
%
% Why 5: ~10% of the 51 frames, in line with the standard 10{20% hold-out
% convention.
nQuarantine = 5;
quarantineIdx = sort(randperm(numFrames, nQuarantine));
developIdx    = setdiff(1:numFrames, quarantineIdx);

fprintf('KF train indices (MATLAB) : %s\n', mat2str(trainIdx));
fprintf('KF test indices (MATLAB) : %s\n', mat2str(testIdx));
fprintf('Shape/HoG quarantine (1..%d): %s\n', numFrames, mat2str(quarantineIdx));

% ---- 0.4 Sanity view: overlay masks on a few sample frames -----
% First, middle and last frames | chosen for visual coverage of the
% sequence, not used by any subsequent computation. Display only.
sampleIdx = unique([1, round(numFrames/2), numFrames]);
figure('Name', 'STEP 0 sanity: mask overlay on RGB');
tiledlayout(1, numel(sampleIdx), 'Padding', 'compact', 'TileSpacing', 'compact');
for k = 1:numel(sampleIdx)
    i = sampleIdx(k);
    nexttile;
    imshow(labeloverlay(rgbImgs{i}, gtMasks{i}, ...
        'Colormap', [1 0 0], 'Transparency', 0.55));
    title(sprintf('Frame %d (brief idx %d)', i, i-1));
end

%% =====
% STEP 1: Task 1(i) | Shape features
% For each GT mask compute four shape descriptors:
%     solidity      = Area / ConvexArea           in (0, 1]
%     circularity   = 4*pi*Area / Perimeter^2     in (0, 1]
%     non_compactness = 1 - circularity           in [0, 1)
%     eccentricity  = ellipse eccentricity of the region in [0, 1)
% All four are scale- and translation-invariant, so they depend on
% the parachute's shape alone and not on where it sits in the frame.
%% =====

```

```

% ---- 1.1 Compute the four features for every frame -----
shapeFeatures = zeros(numFrames, 4); % columns: [sol, nc, circ, ecc]
shapeNames     = {'solidity', 'non-compactness', 'circularity', 'eccentricity'};

for i = 1:numFrames
    shapeFeatures(i, :) = computeShapeFeatures(gtMasks{i});
end

% Save as a table for readability (frame index column uses the brief's
% 0-based numbering, which is how every plot/table in the report refers
% to frames).
shapeTable = array2table(shapeFeatures, 'VariableNames', ...
    {'solidity', 'non_compactness', 'circularity', 'eccentricity'});
shapeTable.frame = (0:numFrames-1)';
shapeTable = movevars(shapeTable, 'frame', 'Before', 1);

% ---- 1.2 Plot each feature's trajectory across the sequence -----
frames0 = 0:numFrames-1; % brief-numbered frame axis
figure('Name', 'STEP 1: Shape-feature trajectories');
tiledlayout(2, 2, 'Padding', 'compact', 'TileSpacing', 'compact');
for j = 1:4
    nexttile;
    plot(frames0, shapeFeatures(:, j), '-o', 'LineWidth', 1.2, 'MarkerSize', 3);
    grid on;
    xlabel('frame index (brief numbering)');
    ylabel(shapeNames{j});
    title(shapeNames{j});
end
sgtitle('Shape features across the 51 parachute frames');

% ---- 1.3 Summary statistics (develop set vs. quarantine) -----
% any commentary we write about trajectories is grounded
% in the development subset; the quarantine numbers are the independent
% confirmation.
fprintf('\n--- STEP 1: Shape-feature summary ---\n');
fprintf('%-16s %10s %10s | %10s %10s\n', ...
    'feature', 'dev mean', 'dev std', 'qtn mean', 'qtn std');
for j = 1:4
    devVals = shapeFeatures(developIdx, j);
    qtnVals = shapeFeatures(quarantineIdx, j);
    fprintf('%-16s %10.4f %10.4f | %10.4f %10.4f\n', ...
        shapeNames{j}, mean(devVals), std(devVals), ...
        mean(qtnVals), std(qtnVals));
end

% Expose for later cells / the discriminative analysis in STEP 3.

%% =====
% STEP 2: Task 1(ii) | HoG features (4 orientation bins)
% One HoG feature vector per image. The orientation histogram uses
% exactly four bins centred at 0°, 45°, 90°, 135° (per the brief).
%
% Input image: the RGB image with the background suppressed by the

```

```

% GT mask, so HoG describes edges on the parachute itself rather
% than sky/cloud texture. (The brief notes that the masks "can be
% used to extract the corresponding areas from the RGB images".)
%
% Cell size: derived from the parachute's bounding-box in each frame
% rather than a fixed pixel value. The DESIGN choice is "how many
% tiles cover the object" (TILES_ACROSS_OBJECT, conventionally 4
% following the SIFT 4x4 grid). The pixel cell size is a derived
% consequence of that choice and the object's apparent size.
%% =====

% ---- 2.1 Compute HoG vectors for every frame -----
% TILES_ACROSS_OBJECT: SIFT-style 4x4 grid convention. Citable design
% choice, NOT a tuned pixel value.
TILES_ACROSS_OBJECT = 4;

% MIN_CELL_PX: mathematical sanity floor. A 4-bin orientation histogram
% built from fewer than ~4 gradient samples is statistically meaningless.
% Below 2 px per cell each cell holds at most 4 gradients, so 2 is the
% smallest cell size at which the histogram has any informational content.
MIN_CELL_PX = 2;

hogVectors = cell(numFrames, 1); % HoG feature vector per frame
hogCellSizes = zeros(numFrames, 1); % the per-frame cell size actually used
hogGridSizes = zeros(numFrames, 2); % the per-frame [nRowsTiles nColsTiles]

for i = 1:numFrames
    [hogVectors{i}, hogCellSizes(i), hogGridSizes(i,:)] = ...
        computeHoG4(rgbImgs{i}, gtMasks{i}, ...
            TILES_ACROSS_OBJECT, MIN_CELL_PX);
end

% Per-frame vector length (varies with bbox aspect ratio, since each cell
% contributes 4 bins and the grid is mxn cells).
hogLengths = cellfun(@numel, hogVectors);

fprintf('\n--- STEP 2: HoG summary ---\n');
fprintf('Cell size (px) : min=%d median=%d max=%d\n', ...
    min(hogCellSizes), round(median(hogCellSizes)), max(hogCellSizes));
fprintf('Grid (rowsxcols) : median=%d x %d\n', ...
    round(median(hogGridSizes(:,1))), round(median(hogGridSizes(:,2))));
fprintf('Vector length : min=%d median=%d max=%d\n', ...
    min(hogLengths), round(median(hogLengths)), max(hogLengths));

% ---- 2.2 Per-bin energy across the sequence -----
% For each frame, sum the contributions in each of the four orientation
% bins (0°, 45°, 90°, 135°). This collapses the variable-length per-frame
% vectors into a fixed 51x4 matrix that's directly plottable, and answers
% "how does the dominant edge orientation change across the sequence?".
binEnergy = zeros(numFrames, 4);
for i = 1:numFrames
    v = hogVectors{i};
    % Vector layout from computeHoG4: [c1_b1 c1_b2 c1_b3 c1_b4 c2_b1 ...]
    % so reshaping into 4 rows groups all values from each bin together.
    binEnergy(i, :) = sum(reshape(v, 4, []), 2)';
end

```

```

end

binNames = {'0°', '45°', '90°', '135°'};

frames0 = 0:numFrames-1;
figure('Name', 'STEP 2: HoG per-bin energy across the sequence');
plot(frames0, binEnergy, '-o', 'LineWidth', 1.2, 'MarkerSize', 3);
grid on; xlabel('frame index (brief numbering)');
ylabel('summed bin energy (L2-normalised vector)');
legend(binNames, 'Location', 'best');
title('HoG: orientation-bin energy per frame');

% ---- 2.3 Heatmap of full HoG vectors (zero-padded) -----
% Different frames produce different-length vectors, so to visualise all
% 51 side-by-side we zero-pad up to the longest. This is display-only.
maxLen = max(hogLengths);
H = zeros(numFrames, maxLen);
for i = 1:numFrames
    v = hogVectors{i};
    H(i, 1:numel(v)) = v;
end

figure('Name', 'STEP 2: HoG vectors heatmap');
imagesc(H); colormap parula; colorbar;
xlabel('HoG vector index (zero-padded to max length)');
ylabel('frame index (MATLAB 1-based)');
title('HoG feature vectors | one row per frame');

% ---- 2.4 Dev vs. quarantine summary -----
fprintf('\nHoG bin-energy summary (dev vs quarantine):\n');
fprintf('%-6s %10s %10s | %10s %10s\n', ...
    'bin', 'dev mean', 'dev std', 'qtn mean', 'qtn std');
for j = 1:4
    devVals = binEnergy(developIdx, j);
    qtnVals = binEnergy(quarantineIdx, j);
    fprintf('%-6s %10.4f %10.4f | %10.4f %10.4f\n', ...
        binNames{j}, mean(devVals), std(devVals), ...
        mean(qtnVals), std(qtnVals));
end

%% =====
% STEP 3: Task 1(iii) | Discriminative comparison
% Quantify how strongly each feature family changes across the
% sequence, so the report's discussion of "which features best
% represent a descending parachute" rests on numbers rather than
% opinion.
%
% Two complementary measures, both computed on the development
% subset and confirmed on the quarantine subset:
%
% (a) Coefficient of variation (CoV) = std / |mean|
%      Unitless. A larger CoV means the feature varies more

```

```

%         relative to its own scale across the sequence | i.e. it
%         carries more temporal information.
%
% (b) Monotonic-trend strength (Spearman rank correlation
%     between feature value and frame index, magnitude).
%     Captures "is there a clear progression across frames?"
%     independent of whether the trend is linear.
%% =====

% ---- 3.1 Build the comparison matrix -----
% Stack the four shape features and the four HoG bin energies side by
% side, frame-by-frame. All eight columns are then comparable on the
% same time axis.
allFeatures = [shapeFeatures, binEnergy];      % 51 × 8
allNames    = [shapeNames,   binNames];      % 1 × 8 cell

nFeat = numel(allNames);

% ---- 3.2 Coefficient of variation (development subset) -----
% A single number per feature: how much it wiggles relative to its own
% scale. Magnitude of the mean prevents a sign flip near zero from
% inflating the ratio artificially.
covDev = std(allFeatures(developIdx, :), 0, 1) ./ ...
        (abs(mean(allFeatures(developIdx, :), 1)) + eps);

covQtn = std(allFeatures(quarantineIdx, :), 0, 1) ./ ...
        (abs(mean(allFeatures(quarantineIdx, :), 1)) + eps);

trendDev = zeros(1, nFeat);
trendQtn = zeros(1, nFeat);
for k = 1:nFeat
    trendDev(k) = abs(spearmanRho(allFeatures(developIdx, k), developIdx(:)));
    trendQtn(k) = abs(spearmanRho(allFeatures(quarantineIdx, k), quarantineIdx(:)));
end

% ---- 3.4 Print the comparison table -----
fprintf('\n--- STEP 3: Discriminative comparison ---\n');
fprintf('%-18s %10s %10s %10s %10s\n', ...
    'feature', 'CoV (dev)', 'CoV (qtn)', '|| (dev)', '|| (qtn)');
fprintf('%s\n', repmat('-', 1, 64));
for k = 1:nFeat
    fprintf('%-18s %10.4f %10.4f %10.4f %10.4f\n', ...
        allNames{k}, covDev(k), covQtn(k), trendDev(k), trendQtn(k));
end

% ---- 3.5 Visualise the comparison -----
% Two grouped bar charts side by side: one for CoV, one for trend
% strength. Dev vs quarantine bars per feature.
figure('Name', 'STEP 3: Feature comparison (CoV and trend strength)');
tiledlayout(1, 2, 'Padding', 'compact', 'TileSpacing', 'compact');

nexttile;
bar([covDev; covQtn]', 'grouped');
set(gca, 'XTickLabel', allNames, 'XTickLabelRotation', 30);
ylabel('coefficient of variation');
title('Variability across the sequence');

```

```

legend({'develop set', 'quarantine set'}, 'Location', 'best');
grid on;

nexttile;
bar([trendDev; trendQtn]', 'grouped');
set(gca, 'XTickLabel', allNames, 'XTickLabelRotation', 30);
ylabel('|Spearman \rho| vs frame index');
title('Monotonic-trend strength');
legend({'develop set', 'quarantine set'}, 'Location', 'best');
grid on;

% ---- 3.6 Headline findings -----
% Pull out the strongest single feature in each family by trend strength,
% so the report's conclusion can name them explicitly without us having
% to re-eyeball the bar chart.
shapeIdx = 1:4;
hogIdx = 5:8;

[~, bestShape] = max(trendDev(shapeIdx));
[~, bestHoG] = max(trendDev(hogIdx));
bestHoG = bestHoG + 4; % shift back into the combined indexing

fprintf('\nStrongest temporal trend in each family (development set):\n');
fprintf(' Shape : %-16s || = %.3f\n', allNames{bestShape}, trendDev(bestShape));
fprintf(' HoG : %-16s || = %.3f\n', allNames{bestHoG}, trendDev(bestHoG));

% ---- 3.7 Multi-seed robustness diagnostic -----
% Purpose: characterise how sensitive the quarantine-side comparison
% statistics are to *which* 5 frames happen to be held out. The fixed
% rng(42) split (purpose: prevent test leakage) remains untouched |
% this block is purely an additional generalisation check.
%
% Method: repeated random sub-sampling. Draw N independent quarantine
% sets of size nQuarantine, recompute CoV and || on each, and report
% the mean ± std across the N runs.
%
% Note: dev-set statistics are computed on ~46 frames each run and are
% very stable; the wobble lives almost entirely in the quarantine side,
% which is the small-sample subset. This block therefore reports
% quarantine-side variability only.

N_SEEDS = 10; % small N: this is a robustness check, not a bootstrap
              % confidence interval. 10 runs is enough to see whether
              % the headline ranking is stable across splits.

covQtnRuns = zeros(N_SEEDS, nFeat);
trendQtnRuns = zeros(N_SEEDS, nFeat);
bestShapeRuns = strings(N_SEEDS, 1);
bestHoGRuns = strings(N_SEEDS, 1);

% Save and restore the global RNG state so this diagnostic does NOT
% perturb the reproducibility of any later step (Kalman filter etc.).
rngState = rng;

for s = 1:N_SEEDS

```



```

rng(1000 + s); % deterministic, distinct from the main rng(42)

qIdx = sort(randperm(numFrames, nQuarantine));

covQtnRuns(s, :) = std(allFeatures(qIdx, :), 0, 1) ./ ...
    (abs(mean(allFeatures(qIdx, :), 1)) + eps);

for k = 1:nFeat
    trendQtnRuns(s, k) = abs(spearmanRho( ...
        allFeatures(qIdx, k), qIdx(:)));
end

[~, bs] = max(trendQtnRuns(s, shapeIdx));
[~, bh] = max(trendQtnRuns(s, hogIdx));
bestShapeRuns(s) = string(allNames{bs});
bestHoGRuns(s) = string(allNames{bh + 4});
end

rng(rngState); % restore original RNG state

fprintf('\n--- STEP 3.7: Multi-seed quarantine-side robustness ---\n');
fprintf('( %d random sub-samples of %d frames each)\n', N_SEEDS, nQuarantine);
fprintf('%-18s %14s %14s\n', 'feature', 'CoV mean±std', '|| mean±std');
fprintf('%s\n', repmat('-', 1, 56));
for k = 1:nFeat
    fprintf('%-18s %6.3f ± %5.3f %6.3f ± %5.3f\n', allNames{k}, ...
        mean(covQtnRuns(:, k)), std(covQtnRuns(:, k)), ...
        mean(trendQtnRuns(:, k)), std(trendQtnRuns(:, k)));
end

% Stability of the headline conclusion: how often does each family's
% "winner by trend strength" come out the same across the seeds?
shapeWinCounts = countByCategory(bestShapeRuns);
hogWinCounts = countByCategory(bestHoGRuns);

fprintf('\nFamily-winner stability across %d sub-samples:\n', N_SEEDS);
fprintf(' Shape : '); printCounts(shapeWinCounts);
fprintf(' HoG : '); printCounts(hogWinCounts);

%% =====
% STEP 4: Task 2 | Measurements from GT masks (centroid + angle)
% For every frame extract two measurements that the Kalman filters
% in STEPS 5{6 will operate on:
%
% (1) Centroid (cx, cy) | mean pixel position of the foreground.
% (2) Orientation from PCA of the foreground pixel coordinates:
% the principal eigenvector of the pixel covariance matrix is
% the direction of greatest spread; its angle is .
%
% PCA returns in [-90°, +90°), but the underlying axis is a line
% with no head/tail | so and + 180° describe the same physical
% orientation. Tiny pixel-level changes between frames can flip
% the sign of the eigenvector and cause spurious 180° jumps. We
% unwrap by chaining: if |t - t-1| > 90°, add/subtract 180°

```

```

% to bring _t closest to _{t-1}. The 90° threshold is a
% geometric midpoint, not a tuned value: any frame-to-frame change
% greater than 90° must be a flip (real rotation between
% consecutive frames is much smaller than that).
%% =====

% ---- 4.1 Per-frame centroid and raw PCA angle -----
centroids = zeros(numFrames, 2); % columns: [cx, cy] in pixels
anglesRaw = zeros(numFrames, 1); % degrees, in [-90, +90), unwrapped LATER

for i = 1:numFrames
    [centroids(i, :), anglesRaw(i)] = measureMask(gtMasks{i});
end

% ---- 4.2 Unwrap the orientation sequence -----
% Chain: each angle is brought to within 90° of its predecessor by
% adding or subtracting 180° as needed. Acts only on the recorded
% representation | does not change the underlying axis being measured.
anglesUnwrapped = anglesRaw;
for i = 2:numFrames
    delta = anglesUnwrapped(i) - anglesUnwrapped(i-1);
    if delta > 90
        anglesUnwrapped(i) = anglesUnwrapped(i) - 180;
    elseif delta < -90
        anglesUnwrapped(i) = anglesUnwrapped(i) + 180;
    end
end

% ---- 4.3 Visualise centroid trajectory and orientation -----
frames0 = 0:numFrames-1;

figure('Name', 'STEP 4: Centroid trajectory and orientation');
tiledlayout(2, 2, 'Padding', 'compact', 'TileSpacing', 'compact');

% (a) Spatial path of the centroid, overlaid on the first RGB frame
nexttile;
imshow(rgbImgs{1}); hold on;
plot(centroids(:,1), centroids(:,2), '-', 'LineWidth', 1.2, 'Color', [1 0.6 0]);
plot(centroids(trainIdx, 1), centroids(trainIdx, 2), 'o', ...
    'MarkerSize', 4, 'MarkerFaceColor', [0 0.6 1], 'MarkerEdgeColor', 'none');
plot(centroids(testIdx, 1), centroids(testIdx, 2), 's', ...
    'MarkerSize', 5, 'MarkerFaceColor', [1 0 0], 'MarkerEdgeColor', 'none');
legend({'path', 'train (0{40})', 'test (41{50})'}, ...
    'Location', 'best', 'TextColor', 'k');
title('Centroid trajectory');
hold off;

% (b) cx vs frame
nexttile;
plot(frames0, centroids(:,1), '-o', 'LineWidth', 1.2, 'MarkerSize', 3);
grid on; xlabel('frame index (brief numbering)');
ylabel('c_x (pixels)');
title('Centroid x-coordinate');

```

```

% (c) cy vs frame
nexttile;
plot(frames0, centroids(:,2), '-o', 'LineWidth', 1.2, 'MarkerSize', 3);
grid on; xlabel('frame index (brief numbering)');
ylabel('c_y (pixels)');
title('Centroid y-coordinate');

% (d) Orientation: raw vs unwrapped
nexttile;
plot(frames0, anglesRaw, 'o-', 'LineWidth', 1.0, 'MarkerSize', 3, ...
      'Color', [0.6 0.6 0.6], 'DisplayName', 'raw (PCA, ±90°)');
hold on;
plot(frames0, anglesUnwrapped, 's-', 'LineWidth', 1.2, 'MarkerSize', 3, ...
      'Color', [0 0.6 1], 'DisplayName', 'unwrapped');
grid on; xlabel('frame index (brief numbering)');
ylabel('orientation \theta (degrees)');
title('Orientation: raw vs unwrapped');
legend('Location', 'best');
hold off;

% ---- 4.4 Summary numbers -----
fprintf('\n--- STEP 4: Measurement summary ---\n');
fprintf('Centroid c_x : range [%.1f, %.1f] (mean %.1f)\n', ...
        min(centroids(:,1)), max(centroids(:,1)), mean(centroids(:,1)));
fprintf('Centroid c_y : range [%.1f, %.1f] (mean %.1f)\n', ...
        min(centroids(:,2)), max(centroids(:,2)), mean(centroids(:,2)));
fprintf('Orientation : raw range [%.1f°, %.1f°] unwrapped range [%.1f°, %.1f°]\n', ...
        min(anglesRaw), max(anglesRaw), min(anglesUnwrapped), max(anglesUnwrapped));

% Number of unwrap corrections applied (a useful diagnostic | if every
% other frame triggered an unwrap, that would suggest the threshold or
% the orientation method needs revisiting; for a clean sequence we
% expect very few).
nFlips = sum(abs(diff(anglesRaw)) > 90);
fprintf('Unwrap corrections applied: %d (out of %d frame transitions)\n', ...
        nFlips, numFrames - 1);

%% =====
% STEP 5: Task 2 | Translation Kalman filter (from scratch)
%
% State vector  $x = [cx, cy, vx, vy]^T$  (4×1)
% Measurement  $z = [cx, cy]^T$  (2×1)
%
% Motion model: constant velocity
%  $x_{\{t|t-1\}} = F * x_{\{t-1|t-1\}}$  F: 4×4
%
% Measurement model:
%  $z_t = H * x_t + \text{noise}$  H: 2×4
%
% Noise covariances:
% Q : process noise (4×4) | derived from frame-to-frame
% velocity changes on TRAIN indices.
% R : measurement noise (2×2) | derived from residuals of

```

```

%                                centroid measurements against a
%                                linear trajectory fit on TRAIN.
%
% Training set = MATLAB indices 1..41    (brief frames 0..40)
% Test set      = MATLAB indices 42..51  (brief frames 41..50)
%
% Testing protocol: after frame 41 (end of training) the filter
% PREDICTS forward 10 steps without consuming any measurements.
% The predicted centroids are compared against the true test-frame
% centroids to compute per-frame Euclidean errors in STEP 7.
%% =====

% ---- 5.1 Build F and H (deterministic, not tuned) -----
% Constant-velocity state-transition: per-frame step dt = 1.
F_trans = [1 0 1 0;
           0 1 0 1;
           0 0 1 0;
           0 0 0 1];

% Measurement model: observe position only.
H_trans = [1 0 0 0;
           0 1 0 0];

% ---- 5.2 Derive R from training-set measurement residuals -----
% "How noisy is the centroid measurement?" | measured empirically.
% Fit a simple linear trajectory  $cx(t) = a + b*t$  and  $cy(t) = a + b*t$  on
% the training indices, treat the residuals as the centroid measurement
% noise, and use their variances as R's diagonal.
t_train = (trainIdx - trainIdx(1))'; % 0, 1, 2, ... for stability
cx_train = centroids(trainIdx, 1);
cy_train = centroids(trainIdx, 2);

% Linear fit: [ones t] * [a; b] coord → solve with backslash.
A_fit = [ones(numel(t_train), 1), t_train];
px = A_fit \ cx_train;
py = A_fit \ cy_train;

cx_resid = cx_train - A_fit * px;
cy_resid = cy_train - A_fit * py;

R_trans = diag([var(cx_resid), var(cy_resid)]);

% ---- 5.3 Derive Q from training-set velocity changes -----
% "How wrong is the constant-velocity assumption?" | measured empirically.
% Compute frame-to-frame position differences (= empirical velocity),
% then the differences of those differences (= empirical velocity
% changes). The variance of velocity changes sizes the velocity part of
% Q; divide by dt^2 → dt^4 factors for the standard continuous-white-
% noise-acceleration discretisation. With dt = 1 frame, the factors are 1.
vel_train_x = diff(cx_train); % empirical vx
vel_train_y = diff(cy_train); % empirical vy
accel_x      = diff(vel_train_x); % frame-to-frame vx change
accel_y      = diff(vel_train_y); % frame-to-frame vy change

```

```

sigma_ax2 = var(accel_x);
sigma_ay2 = var(accel_y);

dt = 1;
Q_base = @(s2) s2 * [dt^4/4  0      dt^3/2  0;
                    0      dt^4/4  0      dt^3/2;
                    dt^3/2  0      dt^2    0;
                    0      dt^3/2  0      dt^2];

Q_trans = blkdiag([sigma_ax2 * dt^4/4, sigma_ax2 * dt^3/2;
                  sigma_ax2 * dt^3/2, sigma_ax2 * dt^2], ...
                  [sigma_ay2 * dt^4/4, sigma_ay2 * dt^3/2;
                  sigma_ay2 * dt^3/2, sigma_ay2 * dt^2]);

% The blkdiag above orders states [cx vx cy vy]; reorder to match our
% [cx cy vx vy] convention. A small explicit permutation is clearer than
% rebuilding the block matrix by hand.
perm = [1 3 2 4];
Q_trans = Q_trans(perm, perm);

% ---- 5.4 Initial state x0 and initial covariance P0 -----
% x0: first measured centroid; initial velocity = difference between the
% first two measured centroids (simplest data-driven estimate).
x0_trans = [centroids(1, 1);
            centroids(1, 2);
            centroids(2, 1) - centroids(1, 1);
            centroids(2, 2) - centroids(1, 2)];

% P0: initial uncertainty. We use the variance of centroids across the
% training set as a scale | large enough that the filter weights early
% measurements strongly (rather than trusting x0 blindly), but bounded
% so updates remain numerically well-posed.
P0_trans = diag([var(cx_train), var(cy_train), ...
                var(vel_train_x), var(vel_train_y)]);

% ---- 5.5 Run filter: predict+update on TRAIN, predict-only on TEST ----
% Storage: we record, for every frame, the posterior state (after any
% update that happened at that frame) AND the pure prediction (before
% the update). For test frames the "prediction" is what we will compare
% to GT in STEP 7; there is no update step there.

nState = 4;
xHist   = zeros(nState, numFrames); % posterior (or prediction) state
xPred   = zeros(nState, numFrames); % one-step-ahead prediction state

x = x0_trans;
P = P0_trans;

% First frame (t = 1): the measurement is the centroid at frame 1.
% Update with it so the estimate at frame 1 reflects both x0 and z1.
z = centroids(1, :)';
[x, P] = kfUpdate(x, P, z, H_trans, R_trans);
xHist(:, 1) = x;
xPred(:, 1) = x; % no prior prediction for frame 1

```

```

for i = 2:numFrames
    % --- Predict ---
    [xp, Pp] = kfPredict(x, P, F_trans, Q_trans);
    xPred(:, i) = xp;

    if ismember(i, trainIdx)
        % Training frame: consume the measurement.
        z = centroids(i, :);
        [x, P] = kfUpdate(xp, Pp, z, H_trans, R_trans);
    else
        % Test frame: no measurement | posterior = prediction.
        x = xp;
        P = Pp;
    end
    xHist(:, i) = x;
end

% Extract predicted centroids for the test frames (these are the
% numbers STEP 7 compares against GT).
kfCentroids      = xHist(1:2, :);      % 51 x 2, posterior/prediction
kfPredCentroids  = xPred(1:2, :);      % 51 x 2, pure one-step predictions

% ---- 5.6 Visualise and report -----
figure('Name', 'STEP 5: Translation KF | estimated vs measured');
tiledlayout(2, 2, 'Padding', 'compact', 'TileSpacing', 'compact');

% (a) Spatial path: measured vs filtered vs predicted, on the first RGB.
% GT line is drawn LAST and thin so it sits on top of the markers
% without obscuring them | confirms visually how closely the markers
% sit on the GT path.
nexttile;
imshow(rgbImgs{1}); hold on;
plot(kfCentroids(trainIdx, 1), kfCentroids(trainIdx, 2), 'o', ...
     'MarkerSize', 4, 'MarkerFaceColor', [0 0.6 1], ...
     'MarkerEdgeColor', 'none', 'DisplayName', 'KF posterior (train)');
plot(kfCentroids(testIdx, 1), kfCentroids(testIdx, 2), 's', ...
     'MarkerSize', 5, 'MarkerFaceColor', [1 0 0], ...
     'MarkerEdgeColor', 'none', 'DisplayName', 'KF prediction (test)');
plot(centroids(:,1), centroids(:,2), '-', 'LineWidth', 0.7, ...
     'Color', [1 0.6 0], 'DisplayName', 'GT centroid');
legend('Location', 'best', 'TextColor', 'k');
title('Centroid trajectory | measured vs filtered');
hold off;

% (b) cx vs frame
nexttile;
plot(frames0, centroids(:,1), 'o-', 'LineWidth', 1.0, 'MarkerSize', 3, ...
     'Color', [1 0.6 0], 'DisplayName', 'GT'); hold on;
plot(frames0, kfCentroids(:,1), '.-', 'LineWidth', 1.2, ...
     'Color', [0 0.6 1], 'DisplayName', 'KF');
xl = xline(testIdx(1) - 1, '--', 'Color', [0.6 0.6 0.6], ...
     'Label', 'test frames →', 'LabelHorizontalAlignment', 'right');
xl.Annotation.LegendInformation.IconDisplayStyle = 'off';
grid on; xlabel('frame index (brief numbering)');

```

```

ylabel('c_x (pixels)');
title('c_x: measured vs KF'); legend('Location', 'best');
hold off;

% (c) cy vs frame
nexttile;
plot(frames0, centroids(:,2), 'o-', 'LineWidth', 1.0, 'MarkerSize', 3, ...
      'Color', [1 0.6 0], 'DisplayName', 'GT'); hold on;
plot(frames0, kfCentroids(:,2), '.-', 'LineWidth', 1.2, ...
      'Color', [0 0.6 1], 'DisplayName', 'KF');
xl = xline(testIdx(1) - 1, '--', 'Color', [0.6 0.6 0.6], ...
          'Label', 'test frames →', 'LabelHorizontalAlignment', 'right');
xl.Annotation.LegendInformation.IconDisplayStyle = 'off';
grid on; xlabel('frame index (brief numbering)');
ylabel('c_y (pixels)');
title('c_y: measured vs KF'); legend('Location', 'best');
hold off;

% (d) Velocity estimates from the filter state (diagnostic only)
nexttile;
plot(frames0, xHist(3,:), '-', 'LineWidth', 1.2, 'DisplayName', 'v_x'); hold on;
plot(frames0, xHist(4,:), '-', 'LineWidth', 1.2, 'DisplayName', 'v_y');
xl = xline(testIdx(1) - 1, '--', 'Color', [0.6 0.6 0.6]);
xl.Annotation.LegendInformation.IconDisplayStyle = 'off';
grid on; xlabel('frame index (brief numbering)');
ylabel('velocity (pixels/frame)');
title('Filter velocity state'); legend('Location', 'best');
hold off;

% ---- 5.7 Print noise covariances and a summary -----
fprintf('\n--- STEP 5: Translation Kalman filter ---\n');
fprintf('R (measurement noise, 2x2):\n');
disp(R_trans);
fprintf('Q (process noise, 4x4, state order [cx cy vx vy]):\n');
disp(Q_trans);
fprintf('x0 = [%2f, %2f, %2f, %2f] (cx, cy, vx, vy)\n', x0_trans);

% Quick sanity numbers | full error evaluation lives in STEP 7.
posErr_train = sqrt(sum((kfCentroids(trainIdx, :) - centroids(trainIdx, :)).^2, 2));
posErr_test = sqrt(sum((kfCentroids(testIdx, :) - centroids(testIdx, :)).^2, 2));
fprintf('Train-frame posterior-vs-GT RMS error : %2f px\n', ...
        sqrt(mean(posErr_train.^2)));
fprintf('Test-frame prediction-vs-GT RMS error : %2f px (10 frames, prediction only)\n',
        sqrt(mean(posErr_test.^2)));

%% =====
% STEP 6: Task 2 | Rotation Kalman filter (from scratch)
%
% State vector x = [theta, omega]^T (2x1)
%   theta : orientation in degrees (unwrapped from STEP 4)
%   omega : angular velocity in degrees per frame
%
% Measurement z = [theta] (1x1)

```

```

%
% Motion model: constant angular velocity (dt = 1)
%     theta_{t} = theta_{t-1} + omega_{t-1}
%     omega_{t} = omega_{t-1}
%
% Same train/test protocol as STEP 5:
%     - Predict + update on training frames (1..41).
%     - Predict only on test frames (42..51), no measurements consumed.
%
% Q and R derived from training-frame data; nothing tuned to the test
% set. The unwrapped sequence from STEP 4 was a clean ~1.5°/frame
% monotone rise, so the constant-angular-velocity assumption is
% on stronger empirical footing than constant velocity was for cy.
%% =====

% ---- 6.1 Build F and H -----
F_rot = [1 1;
         0 1];

H_rot = [1 0];

% ---- 6.2 Derive R from training-frame angle residuals -----
% Same logic as the translation R: linear fit on training frames; the
% variance of residuals against that fit is the angle measurement noise.
theta_train = anglesUnwrapped(trainIdx);

% Local time vector for the linear fit | recomputed here so Step 6 does
% not depend on workspace variables left by Step 5.
t_train_rot = (trainIdx - trainIdx(1))';
A_fit_rot   = [ones(numel(t_train_rot), 1), t_train_rot];
ptheta      = A_fit_rot \ theta_train;

theta_resid = theta_train - A_fit_rot * ptheta;
R_rot = var(theta_resid);

% ---- 6.3 Derive Q from training-frame angular-velocity changes -----
% Same structure as translation Q, scalar version.
omega_train = diff(theta_train);           % empirical omega per frame
alpha_train = diff(omega_train);           % empirical angular accel
sigma_alpha2 = var(alpha_train);

Q_rot = sigma_alpha2 * [1/4  1/2;
                       1/2  1 ];

% ---- 6.4 Initial state and covariance -----
x0_rot = [theta_train(1);
          theta_train(2) - theta_train(1)];

P0_rot = diag([var(theta_train), var(omega_train)]);

% ---- 6.5 Run the filter -----
xHistRot = zeros(2, numFrames);

```



```

x = x0_rot;
P = P0_rot;

% First-frame update with the first measurement.
z = anglesUnwrapped(1);
[x, P] = kfUpdate(x, P, z, H_rot, R_rot);
xHistRot(:, 1) = x;

for i = 2:numFrames
    [xp, Pp] = kfPredict(x, P, F_rot, Q_rot);
    if ismember(i, trainIdx)
        z = anglesUnwrapped(i);
        [x, P] = kfUpdate(xp, Pp, z, H_rot, R_rot);
    else
        x = xp;
        P = Pp;
    end
    xHistRot(:, i) = x;
end

kfThetas = xHistRot(1, :)' ;
kfOmegas = xHistRot(2, :)' ;

% ---- 6.6 Visualise -----
figure('Name', 'STEP 6: Rotation KF | estimated vs measured');
tiledlayout(1, 2, 'Padding', 'compact', 'TileSpacing', 'compact');

% (a) Theta vs frame, GT vs filter
nexttile;
plot(frames0(trainIdx), kfThetas(trainIdx), 'o', ...
    'MarkerSize', 4, 'MarkerFaceColor', [0 0.6 1], ...
    'MarkerEdgeColor', 'none', 'DisplayName', 'KF posterior (train)');
hold on;
plot(frames0(testIdx), kfThetas(testIdx), 's', ...
    'MarkerSize', 5, 'MarkerFaceColor', [1 0 0], ...
    'MarkerEdgeColor', 'none', 'DisplayName', 'KF prediction (test)');
plot(frames0, anglesUnwrapped, '-', 'LineWidth', 0.7, ...
    'Color', [1 0.6 0], 'DisplayName', 'GT \theta (unwrapped)');
xl = xline(testIdx(1) - 1, '--', 'Color', [0.6 0.6 0.6], ...
    'Label', 'test frames →', 'LabelHorizontalAlignment', 'right');
xl.Annotation.LegendInformation.IconDisplayStyle = 'off';
grid on; xlabel('frame index (brief numbering)');
ylabel('orientation \theta (degrees)');
title('\theta: measured vs KF');
legend('Location', 'best');
hold off;

% (b) Filter angular-velocity state
nexttile;
plot(frames0, kfOmegas, '-', 'LineWidth', 1.2, 'DisplayName', '\omega');
xl = xline(testIdx(1) - 1, '--', 'Color', [0.6 0.6 0.6]);
xl.Annotation.LegendInformation.IconDisplayStyle = 'off';
grid on; xlabel('frame index (brief numbering)');
ylabel('angular velocity (deg/frame)');
title('Filter angular-velocity state');

```

```

legend('Location', 'best');
hold off;

% ---- 6.7 Print noise covariances and a summary -----
fprintf('\n--- STEP 6: Rotation Kalman filter ---\n');
fprintf('R (measurement noise)      : %.4f deg^2   (std %.3f deg)\n', ...
    R_rot, sqrt(R_rot));
fprintf('Q (process noise, 2x2 deg^2):\n');
disp(Q_rot);
fprintf('x0 = [%.2f deg, %.3f deg/frame]   (theta, omega)\n', x0_rot);

% Sanity numbers | full evaluation in STEP 7 with the brief's exact metric.
% Plain residual here for now (raw difference, not yet wrapped).
rawErr_train = abs(kfThetas(trainIdx) - anglesUnwrapped(trainIdx));
rawErr_test  = abs(kfThetas(testIdx) - anglesUnwrapped(testIdx));
fprintf('Train-frame |theta_KF - theta_GT| RMS : %.3f deg\n', ...
    sqrt(mean(rawErr_train.^2)));
fprintf('Test-frame  |theta_KF - theta_GT| RMS : %.3f deg   (10 frames, prediction only)\n', .
    sqrt(mean(rawErr_test.^2)));

%% =====
% STEP 7: Results | per-frame error table and plots
%
% Brief-specified metrics for the 10 prediction frames (test set):
%
% Translation error (Euclidean, in pixels):
%     e_pos(t) = sqrt( (cx_KF - cx_GT)^2 + (cy_KF - cy_GT)^2 )
%
% Rotation error (smallest angular difference, in degrees):
%     e_theta(t) = min( || , 180 - || )      where  = _KF - _GT
%
% The min(||, 180 - ||) form respects the 180° axis ambiguity:
% an estimate of  and  + 180° describe the same physical orientation,
% so the error metric must treat them as equivalent.
%
% Outputs:
% - A frame-by-frame table printed to the Command Window.
% - A two-panel figure with translation and rotation errors vs frame.
% - Summary statistics (mean, max, RMS) for both error series.
%% =====

% ---- 7.1 Per-frame translation error -----
posErr = sqrt( sum( (kfCentroids(testIdx, :) - centroids(testIdx, :)).^2, 2 ) );

% ---- 7.2 Per-frame rotation error using the brief's exact metric -----
% in degrees, then folded onto [0, 90] via min(||, 180 - ||) so
% that two angles 180° apart count as zero error.
rawDelta = kfThetas(testIdx) - anglesUnwrapped(testIdx);
absDelta = mod(abs(rawDelta), 180);      % bring into [0, 180]
rotErr    = min(absDelta, 180 - absDelta); % fold onto [0, 90]

```

```

% ---- 7.3 Per-frame results table -----
testFramesBrief = (testIdx - 1)'; % brief uses 0-based numbering
resultsTable = table( ...
    testFramesBrief, ...
    centroids(testIdx, 1), centroids(testIdx, 2), ...
    kfCentroids(testIdx, 1), kfCentroids(testIdx, 2), ...
    posErr, ...
    anglesUnwrapped(testIdx), kfThetas(testIdx), ...
    rotErr, ...
    'VariableNames', { ...
        'frame', ...
        'cx_GT', 'cy_GT', ...
        'cx_KF', 'cy_KF', ...
        'pos_err_px', ...
        'theta_GT_deg', 'theta_KF_deg', ...
        'rot_err_deg' });

fprintf('\n--- STEP 7: Per-frame errors on test frames (brief frames 41{50} ---\n');
disp(resultsTable);

% ---- 7.4 Summary statistics -----
fprintf('Translation error (pixels):\n');
fprintf('    mean = %.3f    RMS = %.3f    max = %.3f\n', ...
    mean(posErr), sqrt(mean(posErr.^2)), max(posErr));
fprintf('Rotation error (degrees):\n');
fprintf('    mean = %.3f    RMS = %.3f    max = %.3f\n', ...
    mean(rotErr), sqrt(mean(rotErr.^2)), max(rotErr));

% ---- 7.5 Error-vs-frame plots -----
figure('Name', 'STEP 7: Per-frame prediction errors on test set');
tiledlayout(1, 2, 'Padding', 'compact', 'TileSpacing', 'compact');

nexttile;
bar(testFramesBrief, posErr, 'FaceColor', [0 0.6 1], 'EdgeColor', 'none');
hold on;
yline(mean(posErr), '--', 'Color', [0.6 0.6 0.6], ...
    'Label', sprintf('mean %.2f', mean(posErr)), 'LabelHorizontalAlignment', 'left');
yline(sqrt(mean(posErr.^2)), '-', 'Color', [1 0.4 0.4], ...
    'Label', sprintf('RMS %.2f', sqrt(mean(posErr.^2))), 'LabelHorizontalAlignment', 'right');
grid on; xlabel('frame index (brief numbering)');
ylabel('translation error (pixels)');
title('Translation error per test frame');
hold off;

nexttile;
bar(testFramesBrief, rotErr, 'FaceColor', [1 0.6 0], 'EdgeColor', 'none');
hold on;
yline(mean(rotErr), '--', 'Color', [0.6 0.6 0.6], ...
    'Label', sprintf('mean %.2f', mean(rotErr)), 'LabelHorizontalAlignment', 'left');
yline(sqrt(mean(rotErr.^2)), '-', 'Color', [1 0.4 0.4], ...
    'Label', sprintf('RMS %.2f', sqrt(mean(rotErr.^2))), 'LabelHorizontalAlignment', 'right');
grid on; xlabel('frame index (brief numbering)');
ylabel('rotation error (degrees)');
title('Rotation error per test frame');

```

```
hold off;
```

```
%% =====  
% Local helper functions  
%% =====
```

```
function [x, P] = kfPredict(x, P, F, Q)  
% kfPredict Kalman prediction step.  
%  
%  $x = F * x$  (state advances by the motion model)  
%  $P = F * P * F' + Q$  (covariance grows by process noise)  
%  $x = F * x;$   
%  $P = F * P * F' + Q;$   
end
```

```
function [x, P] = kfUpdate(x, P, z, H, R)  
% kfUpdate Kalman measurement-update step.  
%  
%  $y = z - H*x$  (innovation / residual between measurement  
% and predicted measurement)  
%  $S = H*P*H' + R$  (innovation covariance)  
%  $K = P*H'*inv(S)$  (Kalman gain | weights prediction vs measurement  
% by their relative uncertainties)  
%  $x = x + K*y$  (state blended with measurement)  
%  $P = (I - K*H) * P$  (covariance shrinks after absorbing evidence)  
%  
% Joseph-form covariance update is used below for numerical stability:  
% it remains positive-semidefinite even with small floating-point errors.  
%  $y = z - H * x;$   
%  $S = H * P * H' + R;$   
%  $K = P * H' / S;$  % "/" S" is algebraic inverse  
%  $x = x + K * y;$   
%  $I = eye(size(P));$   
%  $KH = K * H;$   
%  $P = (I - KH) * P * (I - KH)' + K * R * K';$  % Joseph form  
end
```

```
function [centroid, angleDeg] = measureMask(mask)  
% measureMask Centroid (mean pixel position) and PCA-based orientation  
% angle of the foreground region in a binary mask.  
%  
% OUTPUTS  
% centroid [cx, cy] in pixel coordinates (image x = column, y = row).  
% angleDeg orientation in degrees, in (-90, +90].  
% Convention: angle of the principal axis measured from the  
% positive x-axis, with +y pointing DOWN (image coordinates).  
%  
% Method  
% 1. Take all foreground pixel coordinates as a 2-D point cloud.  
% 2. Mean-centre them.  
% 3. Compute the 2x2 covariance matrix.  
% 4. Eigendecompose; the eigenvector with the larger eigenvalue is  
% the principal axis (direction of greatest spread).
```

```

% 5. Angle = atan2(eigenvector_y, eigenvector_x).
%
% Domain assumption: a single object per mask (the parachute). If the
% mask contains multiple disconnected components we measure the largest.
mask = logical(mask);
if ~any(mask(:))
    centroid = [NaN NaN];
    angleDeg = NaN;
    return;
end

% Keep only the largest connected component (the parachute).
mask = bwareafilt(mask, 1);

% Pixel coordinates of every foreground pixel.
[yPx, xPx] = find(mask);
pts = [xPx, yPx];

% Centroid = mean coordinate.
centroid = mean(pts, 1);

% PCA: mean-centre, covariance, eigendecomposition.
centred = pts - centroid;
C = (centred' * centred) / size(centred, 1); % 2x2 covariance

[V, D] = eig(C);
eigenvalues = diag(D);

% Principal axis = eigenvector with the larger eigenvalue.
[~, principalIdx] = max(eigenvalues);
principal = V(:, principalIdx); % [vx; vy]

% Angle in degrees from +x axis. atan2 returns (-pi, pi], so the
% output range here is (-180°, 180°]. The 180° axis ambiguity (an
% axis is a line, not an arrow) is resolved later by the
% unwrapping pass in STEP 4.2 | locally we just report whatever
% atan2 gives, then fold to (-90, 90] so unwrapping has a
% canonical starting representation.
angleDeg = atan2d(principal(2), principal(1));

% Fold to (-90, +90] so the raw angle lies in a single half-plane,
% matching the natural ambiguity of an unsigned axis.
if angleDeg > 90
    angleDeg = angleDeg - 180;
elseif angleDeg <= -90
    angleDeg = angleDeg + 180;
end
end

function counts = countByCategory(strArr)
% countByCategory Tally a string array into a Map of category -> count.
% Pure base-MATLAB; avoids the Statistics Toolbox 'tabulate' function.
counts = containers.Map('KeyType', 'char', 'ValueType', 'double');
for i = 1:numel(strArr)
    key = char(strArr(i));

```

```

        if isKey(counts, key)
            counts(key) = counts(key) + 1;
        else
            counts(key) = 1;
        end
    end
end

```

```

function printCounts(counts)
% printCounts One-line "label×N, label×N, ..." formatter for a counts Map.
    keys = counts.keys;
    parts = strings(1, numel(keys));
    for i = 1:numel(keys)
        parts(i) = sprintf('%s × %d', keys{i}, counts(keys{i}));
    end
    fprintf('%s\n', strjoin(parts, ', '));
end

```

```

function rho = spearmanRho(x, y)
% spearmanRho Spearman rank correlation between two vectors, computed
% using only base-MATLAB facilities (no Statistics Toolbox required).
%
% Spearman = Pearson correlation of the ranks of x and y. We use a
% local tied-rank function so that values which happen to be equal get
% the average of the ranks they would otherwise occupy.
    x = x(:); y = y(:);
    rx = tiedRanks(x);
    ry = tiedRanks(y);
    R = corrcoef(rx, ry); % corrcoef is in base MATLAB
    rho = R(1, 2);
end

```

```

function r = tiedRanks(v)
% tiedRanks Rank a vector with mid-rank handling for ties (matches the
% behaviour of the Statistics Toolbox tiedrank function).
%
% Sort the values, give each its position, then for any group of equal
% values average their positions so no rank is "favoured" over another.
    v = v(:);
    n = numel(v);
    [~, ord] = sort(v, 'ascend');
    ranks = zeros(n, 1);
    ranks(ord) = 1:n;

    % Average ranks within tied groups
    [vSorted, ord2] = sort(v, 'ascend');
    i = 1;
    while i <= n
        j = i;
        while j < n && vSorted(j+1) == vSorted(i)
            j = j + 1;
        end
        if j > i

```

```

        ranks(ord2(i:j)) = mean(i:j);
    end
    i = j + 1;
end
r = ranks;
end

```

```

function [vec, cellPx, gridRC] = computeHoG4(rgbImg, mask, tilesAcross, minCellPx)
% computeHoG4 HoG feature with 4 orientation bins (0/45/90/135°),
% computed on the RGB image with the background suppressed by 'mask'.
%
% INPUTS
%   rgbImg      H×W×3 RGB image (uint8 or double).
%   mask        H×W logical mask (true = parachute, false = background).
%   tilesAcross design choice: number of HoG tiles across the object's
%               smaller bounding-box side. Conventionally 4 (SIFT-style).
%   minCellPx   sanity floor on cell side length, in pixels.
%
% OUTPUTS
%   vec         1×N HoG feature vector, L2-normalised.
%               Layout: cells in row-major order, four bins per cell, so
%               vec = [c1_b1 c1_b2 c1_b3 c1_b4 c2_b1 c2_b2 ...].
%   cellPx      scalar cell side length in pixels actually used.
%   gridRC      [nRowsTiles nColsTiles] of the tile grid placed on the bbox.

% --- 1. Convert to greyscale double in [0,1] for gradient stability.
if size(rgbImg, 3) == 3
    gImg = rgb2gray(rgbImg);
else
    gImg = rgbImg;
end
gImg = im2double(gImg);

% --- 2. Apply the mask: zero out background pixels so subsequent
% gradients only carry information about the parachute and its edge.
gImg(~mask) = 0;

% --- 3. Crop to the parachute's bounding box. This makes the
% per-frame "image area for HoG" object-relative, so the same
% grid resolution covers the same fraction of the parachute in
% every frame regardless of where it sits or how large it appears.
stats = regionprops(bwareafilt(mask, 1), 'BoundingBox');
if isempty(stats)
    vec = []; cellPx = NaN; gridRC = [0 0];
    return;
end
bb = stats.BoundingBox; % [x y width height], 0.5-offset
x0 = max(1, floor(bb(1)));
y0 = max(1, floor(bb(2)));
x1 = min(size(gImg, 2), x0 + ceil(bb(3)) - 1);
y1 = min(size(gImg, 1), y0 + ceil(bb(4)) - 1);
patch = gImg(y0:y1, x0:x1);

[hP, wP] = size(patch);

```

```

% --- 4. Choose cell size by dividing the SHORTER bbox side into
% 'tilesAcross' tiles. Using the shorter side guarantees at least
% 'tilesAcross' tiles in both dimensions; the longer side then gets
% more tiles, preserving the bbox aspect ratio in the grid.
cellPx = max(minCellPx, floor(min(hP, wP) / tilesAcross));

nRows = floor(hP / cellPx);
nCols = floor(wP / cellPx);
gridRC = [nRows, nCols];

if nRows < 1 || nCols < 1
    vec = []; return;
end

% Trim the patch to an exact multiple of cellPx so the cells tile cleanly.
patch = patch(1:nRows*cellPx, 1:nCols*cellPx);

% --- 5. Image gradients via centred-difference filter [-1 0 1].
% This is the canonical HoG operator (Dalal & Triggs 2005).
fx = [-1 0 1];
fy = fx';
gx = imfilter(patch, fx, 'replicate');
gy = imfilter(patch, fy, 'replicate');

mag = sqrt(gx.^2 + gy.^2);

% Unsigned orientation in [0, 180): atan2 returns [-pi, pi]; mod by
% pi folds opposite directions onto each other (an edge running NE-SW
% and one running SW-NE are the same edge). Convert to degrees.
ori = mod(atan2(gy, gx), pi) * (180 / pi); % in [0, 180)

% --- 6. Soft-assign each pixel's gradient to one of 4 bins centred
% at 0, 45, 90, 135 degrees (bin width = 45°). Hard-binning is fine
% here | soft (linear-interpolated) binning is a refinement we can
% add later if needed; for 4 bins the difference is small.
binWidth = 180 / 4; % 45° per bin
binIdx = floor(ori / binWidth) + 1; % in {1,2,3,4}
binIdx(binIdx > 4) = 4; % safety cap

% --- 7. Accumulate magnitude into the (cell × bin) histogram.
% Vectorised over all pixels; for each pixel we know its cell row,
% cell column, and bin index, so a single accumarray call builds
% the entire (nRows × nCols × 4) histogram in one pass.
[pyIdx, pxIdx] = ndgrid(1:nRows*cellPx, 1:nCols*cellPx);
cellRow = ceil(pyIdx / cellPx);
cellCol = ceil(pxIdx / cellPx);

subs = [cellRow(:), cellCol(:), binIdx(:)];
hist3D = accumarray(subs, mag(:), [nRows, nCols, 4]);

% --- 8. Flatten in cell-row-major × bin order so the layout is
% [c1_b1 c1_b2 c1_b3 c1_b4 c2_b1 ...] | predictable and simple
% to slice when later analysis needs per-bin or per-cell views.
vec = reshape(permute(hist3D, [3 2 1]), 1, []);

% --- 9. L2-normalise the whole vector once. eps prevents division

```



```

    % by zero on degenerate frames with no gradient (e.g. all-black).
    vec = vec / (norm(vec) + eps);
end

function feats = computeShapeFeatures(mask)
% computeShapeFeatures Return [solidity, non_compactness, circularity,
% eccentricity] for the foreground region of a binary mask.
%
% Formulae (definitions, not tuned values | all dimensionless,
% scale- and translation-invariant):
%   solidity          = Area / ConvexArea
%   circularity       = 4*pi*Area / Perimeter^2          (max = 1, perfect disc)
%   non_compactness   = 1 - circularity
%   eccentricity      = sqrt(1 - (minor/major)^2)        (ellipse eccentricity)
%
% Domain assumption: the dataset contains a single object of interest
% (the parachute), so when the mask has multiple disconnected components
% we keep only the largest | the others are segmentation artefacts.
    mask = logical(mask);
    if ~any(mask(:))
        feats = [NaN NaN NaN NaN];
        return;
    end

    % Keep the largest connected component. The "1" comes from the domain
    % (one parachute per frame), not from tuning.
    mask = bwareafilt(mask, 1);

    stats = regionprops(mask, ...
        'Area', 'Perimeter', 'ConvexArea', 'Eccentricity');

    A = stats.Area;
    P = stats.Perimeter;
    Ac = stats.ConvexArea;
    ecc = stats.Eccentricity;

    % Division-by-zero guard for degenerate masks (single-pixel regions
    % have zero perimeter under MATLAB's estimator).
    if P > 0
        circ = 4 * pi * A / (P^2);
    else
        circ = 0;
    end

    % Mathematical clamp: circularity is provably <= 1 for any planar
    % region (isoperimetric inequality). MATLAB's discrete perimeter
    % estimator can numerically overshoot by a fraction of a percent on
    % small regions, so we cap to the theoretical bound. The "1" here is
    % the bound itself, not a tuned threshold.
    circ = min(circ, 1);

    sol = A / Ac;
    nc = 1 - circ;

    feats = [sol, nc, circ, ecc];

```

```
end
```

```
function mask = extractMask(gtImg)
% extractMask Convert a GT image (either indexed 2-D or RGB "visual"
% version) into a binary parachute mask.
%
% Convention used across this submission:
%     background = 0 (black in either representation)
%     foreground = any non-zero pixel
% The dataset contains a single object of interest (the parachute),
% so collapsing all non-zero labels/colours to TRUE is correct and
% works for both the indexed PNG and the RGB-convenience PNG.
    if ndims(gtImg) == 3
        mask = any(gtImg ~= 0, 3);    % RGB: non-black anywhere in RGB
    else
        mask = gtImg > 0;              % indexed/greyscale: non-zero label
    end
    mask = logical(mask);
end
```

```
function lst = sortByEmbeddedNumber(lst)
% sortByEmbeddedNumber Sort a struct array from dir() by the first
% integer found inside each filename. Falls back to plain lexical
% order if any filename lacks an integer (still deterministic).
    names = {lst.name};
    nums = nan(1, numel(names));
    for k = 1:numel(names)
        tok = regexp(names{k}, '\d+', 'match', 'once');
        if ~isempty(tok)
            nums(k) = str2double(tok);
        end
    end
    if all(~isnan(nums))
        [~, order] = sort(nums);
    else
        [~, order] = sort(names);
    end
    lst = lst(order);
end
```